

Concepts of Programming Languages

Dr. Mohamed Yehia Dahab

A series of horizontal lines in teal and white, located on the right side of the slide, extending from the left edge of the slide.

Chapter 8 Topics

- Introduction
- Selection Statements
- Iterative Statements
- Unconditional Branching
- Conclusions

Levels of Control Flow

- Within expressions
- Among program units
- Among program statements

Control Statements: Evolution

- Much research and argument in the 1960s about the issue
 - One important result: It was proven that all algorithms represented by flowcharts can be coded with only
 - **Two-way selection** and
 - **Pretest logical loops**

Control Structure

- A *control structure* is a control statement and the statements whose execution it controls
- Design question
 - Should a control structure have multiple entries?

Types of Control Structure

- Selection Statements
 - Two-Way Selection Statements
 - Multiple-Way Selection Statements
- Iterative Statements
 - Counter-Controlled Loops
 - Logically-Controlled Loops
 - Pre-test
 - Post-test
 - Iteration Based on Data Structures
- Unconditional Branching

Selection Statements

- A *selection statement* provides the means of choosing between two or more paths of execution
- Two general categories:
 - Two-way selectors
 - Multiple-way selectors

Two-Way Selection Statements

- General form:

```
if control_expression  
    then clause  
    else clause
```
- Design Issues:
 - What is the form and type of the control expression?
 - How are the **then** and **else** clauses specified?
 - How should the meaning of nested selectors be specified?

Two-Way Selection: Examples

- ALGOL 60:
 if (boolean_expr)
 then statement (then clause)
 else statement (else clause)
- The statements could be single or compound

Nesting Selectors

- Java example

```
if (sum == 0)
    if (count == 0)
        result = 0;
    else result = 1;
```

- Which `if` gets the `else`?
- Java's static semantics rule: `else` matches with the nearest `if`

Nesting Selectors (Cont')

- To force an alternative semantics, compound statements may be used:

```
if (sum == 0) {  
    if (count == 0)  
        result = 0;  
}  
else result = 1;
```

- The above solution is used in C, C++, and C#
- Perl requires that all then and else clauses to be compound

Nesting Selectors (Cont')

- Python example

```
if sum == 0:  
    if count == 0:  
        result = 0  
else:  
    result = 1
```

```
if sum == 0:  
    if count == 0:  
        result = 0  
else:  
    result = 1
```

Which `if` gets the `else`?

Multiple-Way Selection Statements

- Allow the selection of one of any number of statements or statement groups
- Design Issues:
 1. What is the form and type of the control expression?
 2. How are the selectable segments specified?
 3. Is execution flow through the structure restricted to include just a single selectable segment?
 4. What is done about unrepresented expression values?

Multiple-Way Selection: Examples

- Early multiple selectors:
 - FORTRAN arithmetic IF (a three-way selector)
IF (arithmetic expression) N1, N2, N3
 - Segments require GOTOs
 - Not encapsulated (selectable segments could be anywhere)

Multiple-Way Selection: Examples

- Modern multiple selectors

- C's `switch` statement

```
switch (expression) {  
    case const_expr_1: stmt_1;  
    ...  
    case const_expr_n: stmt_n;  
    [default: stmt_n+1]  
}
```

Multiple-Way Selection: Examples

- Design choices for C's **switch** statement
 1. Control expression can be only an integer type
 2. Selectable segments can be statement sequences, blocks, or compound statements
 3. Any number of segments can be executed in one execution of the construct (there is no implicit branch at the end of selectable segments)
 4. **default** clause is for unrepresented values (if there is no **default**, the whole statement does nothing)

Multiple-Way Selection: Examples

- The Ada case statement

```
case expression is
    when choice list => stmt_sequence;
    ...
    when choice list => stmt_sequence;
    when others => stmt_sequence;]
end case;
```

- More reliable than C's `switch` (once a `stmt_sequence` execution is completed, control is passed to the first statement after the `case` statement)

Multiple-Way Selection Using `if`

- Multiple Selectors can appear as direct extensions to two-way selectors, using `else-if` clauses, for example in Ada:

```
if ...  
    then ...  
elsif ...  
    then ...  
elsif ...  
    then ...  
    else ...  
end if
```

Iterative Statements

- The repeated execution of a statement or compound statement is accomplished either by iteration or recursion
- General design issues for iteration control statements:
 1. How is iteration controlled?
 2. Where is the control mechanism in the loop?

Counter-Controlled Loops

- A counting iterative statement has a loop variable, and a means of specifying the *initial* and *terminal*, and *stepsize* values
- Design Issues:
 1. What are the type and scope of the loop variable?
 2. What is the value of the loop variable at loop termination?
 3. Should it be legal for the loop variable or loop parameters to be changed in the loop body, and if so, does the change affect loop control?
 4. Should the loop parameters be evaluated only once, or once for every iteration?

Iterative Statements: Examples

- FORTRAN 90 syntax

```
DO label var = start, finish [, stepsize]
```

- Stepsize can be any value but zero
- Parameters can be expressions
- Design choices:
 1. Loop variable must be **INTEGER**
 2. Loop variable always has its last value
 3. The loop variable cannot be changed in the loop, but the parameters can; because they are evaluated only once, it does not affect loop control
 4. Loop parameters are evaluated only once

Iterative Statements: Examples

- **FORTRAN 95 : a second form:**

```
[name:] DO variable = initial, terminal  
[, stepsize]
```

...

```
END DO [name]
```

- Loop variable must be an **INTEGER**

Operational Semantic Description for Fortran “Do” Statement

```
init_value = init_expression
terminal_value = terminal_expression
step_value = step_expression
do_var = init_value
iteration_count =
    max(int((terminal_value - init_value + step_value) / step_value), 0)
loop:
    if iteration_count ≤ 0 goto out
    [loop body]
    do_var = do_var + step_value
    iteration_count = iteration_count - 1
    goto loop
out: ...
```

Iterative Statements

- Pascal's `for` statement

```
for variable := initial (to | downto) final do  
    statement
```

- Design choices:

1. Loop variable must be an ordinal type of usual scope
2. After normal termination, loop variable is undefined
3. The loop variable cannot be changed in the loop; the loop parameters can be changed, but they are evaluated just once, so it does not affect loop control
4. Just once

Iterative Statements: Examples

- Ada

```
for var in [reverse] discrete_range loop
...
end loop
```

- A discrete range is a sub-range of an integer or enumeration type
- Scope of the loop variable is the range of the loop
- Loop variable is implicitly undeclared after loop termination

C's `for` statement : Examples

```
for ([expr_1] ; [expr_2] ; [expr_3]) [statement(s)]
```

- The expressions can be whole statements, or even statement sequences, with the statements separated by commas
- There is no explicit loop variable
- Everything can be changed in the loop
- The first expression is evaluated once, but the other two are evaluated with each iteration

C's for statement : Examples (Cont')

```
for ( ; ; ) ;
```

```
for ( ; ; )  
    printf("Hello!");
```

```
for(int i=0, j=5, k=9; ; ) i++;
```

```
for(Node P=List.getHead(); P!=null; P=P.getNext())  
    printf("%d\n", P.getData());
```

Operational Semantic Description for C “For” Statement

```
    expression_1  
loop:  
    if expression_2 = 0 goto out  
    [loop body]  
    expression_3  
    goto loop  
out: . . .
```

```
for (count1 = 0, count2 = 1.0;  
    count1 <= 10 && count2 <= 100.0;  
    sum = ++count1 + count2, count2 *= 2.5);
```

```
    count1 = 0  
    count2 = 1.0  
loop:  
    if count1 > 10 goto out  
    if count2 > 100.0 goto out  
    count1 = count1 + 1  
    sum = count1 + count2  
    count2 = count2 * 2.5  
    goto loop  
out: ...
```

Python Counter Loop

```
for x in range(5):  
    print (x)
```

```
for x in xrange(30000000):  
    if x == 1:  
        break
```

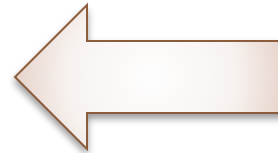
```
for x in range(5):  
    print (x)  
else:  
    print (x+10)
```

Python Counter Loop

```
for num in range(2, 10):  
    for check in range(2, num):  
        if num % check == 0:  
            break  
    else:  
        print(num, 'is prime')
```

Python: List Comprehension

$S = \{x^2 \mid x \in \{0 \dots 9\}\}$
 $T = \{(x, y) \mid x \in S \text{ and } y = x^2\}$
 $W = \{1, 2, 4, 8, \dots, 2^{12}\}$
 $O = \{x \mid x \in S \text{ and } x \text{ even}\}$



Math

Python



$S = [x**2 \text{ for } x \text{ in range}(10)]$
 $S = [x**2 \text{ for } x \text{ in range}(10) \text{ if } x \neq 9 \text{ and } x \neq 7]$
 $T = [(x, x**2) \text{ for } x \text{ in } S]$
 $W = [2**i \text{ for } i \text{ in range}(13)]$
 $O = [x \text{ for } x \text{ in } S \text{ if } x \% 2 == 0]$

Python: List Comprehension (Cont')

Iteration on more than a variable:

*A=[x*y for x in range(3) for y in range(3)]*

A = [0, 0, 0, 0, 1, 2, 0, 2, 4]

Redundant elements

Python: Set Comprehension

These work in a way similar to list comprehensions, but the values are unique (and without sort order) :

*$A = \{x * y \text{ for } x \text{ in range}(3) \text{ for } y \text{ in range}(3)\}$*

$A = \{0, 1, 2, 4\}$

Without redundant elements

The Apply Functions in R As Alternatives To Loops

The **apply()** family appropriate to the **R** to manipulate slices of data from matrices, arrays, lists and dataframes in a repetitive way. These functions allow using the data in a number of ways and avoid explicit use of loop constructs with very few lines of code.

```
X <- c(1,2,3,5,3,5); dim(X) <- c(2,3)
dim(X)
# Sum the values of each column with `apply()`
apply(X, 1, sum)
apply(X, 2, sum)
```

The Apply Functions in R As Alternatives To Loops (Cont')

```
X <- c(1,2,3,5,3,5); dim(X) <- c(2,3)
apply(X, 1, mean)
apply(X, 2, sd)
apply(X, 2, max)
apply(X, 2, median)
apply(X, 2, function (x) length(x)-1)
apply(X, 2, function (x) max(x)+min(x))
apply(X, 2, function (x) max(x)/2+min(x)*3)
```

Iterative Statements: Examples

- C++ differs from C in two ways:
 1. The control expression can also be Boolean
 2. The initial expression can include variable definitions (scope is from the definition to the end of the loop body)
- Java and C#
 - Differs from C++ in that the control expression must be Boolean

Iterative Statements: Logically-Controlled Loops

- Repetition control is based on a Boolean
- Design issues:
 - Pre-test or post-test?
 - Should the logically controlled loop be a special case of the counting loop statement ? expression rather than a counter
- General forms:

while (ctrl_expr_T) loop body	do loop body while (ctrl_expr_T)	do loop body until(ctrl_expr_F)
--	---	--

Iterative Statements: Logically-Controlled Loops: Examples

- Pascal has separate pre-test and post-test logical loop statements (`while-do` and `repeat-until`)
- C and C++ also have both, but the control expression for the post-test version is treated just like in the pre-test case (`while-do` and `do-while`)
- Java is like C, except the control expression must be Boolean (and the body can only be entered at the beginning -- Java has no **goto**)

Iterative Statements: Logically-Controlled Loops: Examples

- Ada has a pretest version, but no post-test
- FORTRAN 77 and 90 have neither
- Perl has two pre-test logical loops, `while` and `until`, but no post-test logical loop

```
#!/usr/local/bin/perl
$a = 3;
until( $a > 5 ){
    printf "a ==> $a\n";
    $a = $a + 1;
}
```

Iterative Statements: User-Located Loop Control Mechanisms

- Sometimes it is convenient for the programmers to decide a location for loop control (other than top or bottom of the loop)
- Simple design for single loops (e.g., `break`)
- Design issues for nested loops
 1. Should the conditional be part of the exit?
 2. Should control be transferable out of more than one loop?

Iterative Statements: User-Located Loop Control Mechanisms `break` and `continue`

- C , C++, and Java: **break** statement
- Unconditional; for any loop or **switch**; one level only
- An alternative: **continue** statement; it skips the remainder of this iteration, but does not exit the loop
- `goto` in **C#** and **break** in **switch** statement